

3.2

- a) Falso. Por convención, los nombres de las funciones empiezan con minúscula (ejemplo: miFuncion). Las mayúsculas iniciales se usan para los nombres de las clases.
- b) Verdadero.
- c) Verdadero.
- d) Falso. Las variables dentro de una función son "locales" y solo funcionan ahí. Los "datos miembros" se declaran en la clase, fuera de las funciones.
- e) Verdadero.
- f) Verdadero.
- g) Verdadero.

3.4

Propósito de un parámetro: Sirve como una variable "comodín" dentro de la función para que esta pueda recibir datos externos y procesarlos. Sin parámetros, las funciones siempre harían lo mismo con los mismos valores.

Diferencia: El parámetro es el nombre que escribes cuando creas la función (el diseño). El argumento es el valor real que le mandas a la función cuando la usas (la ejecución).

3.6

¿Qué es?: Es un constructor que no recibe argumentos. Se utiliza para crear un objeto "base" de una clase sin necesidad de pasarle datos iniciales en ese momento.

Inicialización implícita: Si la clase no tiene un constructor escrito por ti, C++ crea uno automáticamente (implícito). Este constructor llama a los constructores predeterminados de los datos miembros que son objetos, pero deja los tipos básicos (como int, double o bool) con valores basura o no inicializados.

3.8

Encabezado (archivo .h o .hpp):

- **Qué es:** Es un archivo que contiene las declaraciones de las clases, prototipos de funciones y constantes.

- **Propósito:** Sirve como el "índice" o contrato del programa. Permite que otros archivos sepan qué funciones y clases existen sin necesidad de ver todo el código de cómo funcionan por dentro.

Archivo de código fuente (archivo .cpp):

- **Qué es:** Es el archivo donde se escribe la lógica real y el funcionamiento detallado de las funciones y métodos declarados en el encabezado.
- **Propósito:** Contiene las instrucciones que el compilador convertirá en lenguaje de máquina para que el programa se ejecute.

3.10

El propósito de proporcionar funciones establecer (set) y obtener (get) para un dato miembro es implementar el concepto de encapsulamiento. Las razones principales son:

Control y Validación: La función "establecer" permite que la clase valide los datos antes de asignarlos al dato miembro (por ejemplo, asegurarse de que un sueldo no sea negativo), evitando que el objeto tenga un estado inconsistente o incorrecto.

Ocultamiento de datos: Al declarar los datos miembros como private, se protege la integridad de la información. La única forma de interactuar con ellos es a través de estas funciones públicas, lo que hace que el código sea más seguro y fácil de mantener.

Flexibilidad: Permite cambiar la forma interna en que se almacena un dato sin afectar a quienes usan la clase. Solo es necesario actualizar la lógica dentro de las funciones get o set.

Enlace de github

<https://github.com/williamcanton2179/Programacion.git>